Text-to-LoRA: Background, Class-Scale Recreation, and Extensions

Victoria Lee
Deep Learning Midterm Project
`Text-to-LoRA recreation (arXiv:2506.06105)`

Abstract

Large language models can be adapted to specific tasks using methods such as Low-Rank Adaptation (LoRA). Although LoRA updates only a small number of parameters, it still usually requires a separate dataset, training run, and set of tuning decisions for each task. *Text-to-LoRA* (T2L) takes a different approach: it trains a hypernetwork to generate LoRA weights directly from a natural-language description of a task.

This report reviews the original T2L paper, describes a class-scale recreation using a 1.5B-parameter language model, and outlines possible directions for the final project. The recreation implements two training approaches: reconstructing existing task-specific LoRA adapters and training the hypernetwork end-to-end with a language-modeling objective. It also explores the paper's L/M/S hypernetwork variants, Bias-HyperInit, normalized reconstruction targets, and text conditioning with a frozen sentence encoder. Finally, we compare the results with those reported in the paper and identify extensions to investigate in the final project.

1 Introduction

Large language models can perform many tasks, but adapting them to a new task can require substantial time, data, and computing resources. LoRA Hu et al. [2022] makes this process more efficient by keeping the original model weights fixed and training a much smaller set of adapter parameters. However, users still generally need to train a separate adapter for each new task.

Charakorn et al. [2025] ask whether a model could learn how to adapt to tasks in a more general way. Their method, **Text-to-LoRA (T2L)**, uses a hypernetwork Ha et al. [2016] to generate LoRA weights from a written task description. The hypernetwork combines a representation of the task text with information about the model layer and projection type. It then generates the LoRA matrices for the selected query and value projections across the model in one batched forward pass, rather than training a new adapter from scratch for every task.

Goals for this project. This midterm report has three goals: (1) explain the original paper's motivation, method, and main findings; (2) recreate its core approach at a scale that fits the available hardware; and (3) identify extensions to test in the final project.

2 Background: The Original Paper

2.1 Problem setting

Suppose we have a collection of tasks, each with its own training data and a natural-language description. With standard LoRA, we train a separate adapter for each task by minimizing the supervised fine-tuning loss while keeping the pretrained model weights fixed:

Preprint.

$$\Delta W^i = \arg\min_{\Delta W^i} \mathcal{L}_{\text{SFT}}(\mathcal{D}^i, \Psi, \Delta W^i), \tag{1}$$

where Ψ represents the frozen pretrained weights and the LoRA update is written as $\Delta W = BA$, with a low rank r.

T2L replaces the separate adapter-training step with a hypernetwork. The hypernetwork learns parameters θ that map task information to the adapter weights for each module and layer:

$$\Delta W^i_{m,l} = h_\theta(\phi^i_{m,l}), \qquad \phi^i_{m,l} = \text{concat}\big[f(z^i),\, E[m],\, E[l]\big], \tag{2}$$

where $f(z)$ is an embedding of the task description (for example, the CLS representation from `gte-large-en-v1.5`). The learned embeddings $E[m]$ and $E[l]$ identify the module type and layer index. In other words, the hypernetwork uses both the meaning of the task description and the location of the adapter in the model to decide which weights to generate.

2.2 Hypernetwork architectures: L, M, and S

The three T2L variants differ mainly in how the hypernetwork produces the LoRA matrices:

- **L**: the output head generates both A and B together, producing $2rd$ values.
- **M**: a shared output head generates either A or B, depending on an A/B embedding, producing rd values.
- **S**: the head generates one rank vector at a time, using A/B and rank embeddings, and produces d values per output.

All three variants use the same general structure: a task encoder feeds into a mixer MLP, followed by residual MLP blocks and a final output MLP. The paper also uses Bias-HyperInit Beck et al. [2023] to initialize the hypernetwork's output in a way intended to make training more stable. In broad terms, the initialization starts the A-like outputs with a uniform bias and the B-like outputs at zero, with corresponding scaling for the M and S variants.

2.3 Training: reconstruction and supervised fine-tuning

The paper studies two ways to train the hypernetwork.

Reconstruction. The authors begin with a collection of task-specific LoRA adapters, which they treat as reference or "oracle" adapters. They train the hypernetwork to reproduce these existing weights by minimizing the elementwise *L1* distance between its predictions and the reference weights. Before training, the target weights are z-scored, and the predictions are converted back to the original scale at inference time.

This approach shows that a hypernetwork can compress a collection of existing adapters while preserving much of their task performance. However, reproducing known adapters does not necessarily mean the hypernetwork will generate good adapters for new tasks. The paper reports that zero-shot generalization can be weak when tasks with similar descriptions have adapters that are far apart in weight space (Table 6 and Appendix D in the paper).

Supervised fine-tuning (SFT). Instead of asking the hypernetwork to copy existing adapters, the authors can train it through the frozen language model itself. The hypernetwork generates a LoRA adapter from a task description, and the resulting model is optimized using a language-modeling loss. During training, the system samples a task description for each example.

Training on hundreds of Super-NaturalInstructions (SNI) tasks Wang et al. [2022] accessed via HuggingFace allows T2L to generate adapters for tasks it has not specifically trained an adapter for. In the paper's reported experiments, these generated adapters outperform a multi-task LoRA baseline and approach the performance of task-specific oracle adapters on several benchmarks (Table 2).

Table 1: The original paper's setup compared with this class-scale recreation.

	Paper	This project
Base LLM	Mistral-7B-Instruct	Qwen2.5-1.5B-Instruct
Embedder	gte-large-en-v1.5	all-MiniLM-L6-v2
Default variant	M (reported)	M
Oracle tasks	9 benchmarks	4 (BoolQ, ARC-E, PIQA, OpenBookQA)
SFT corpus	479 SNI tasks	16 multi-task slots (synthetic by default; HF/SNI optional)
SFT steps	Up to $\sim$1M	2,500
Hardware	H100-class GPUs	Apple MacBook Pro, 24 GB unified memory, MPS
Descriptions	GPT-4o mini	Templates and paraphrases

2.4 Main findings from the paper

The paper's results point to four main takeaways:

1. A hypernetwork can reproduce a collection of LoRA adapters with some loss in accuracy, making it possible to represent many task-specific adapters compactly.
2. Generating adapters for unseen tasks is harder than reconstructing known ones. Text descriptions are important, and end-to-end SFT performs better for this purpose than reconstruction alone.
3. Increasing the number of training tasks can improve performance, but the training budget also needs to increase enough to learn from the additional tasks.
4. The generated adapters show patterns related to task meaning in activation-space visualizations such as t-SNE. This supports the idea that the task descriptions help organize the generated representations, and that the quality of the descriptions matters.

3 Recreation

3.1 Design constraints and similarity to the original

The original experiments use substantially more computing resources than are available for this class project. We therefore use the 1.5B-parameter Qwen2.5-1.5B-Instruct model, which can run on the available Apple MacBook Pro with 24 GB of unified memory. The goal is to preserve the method's main components while keeping the experiments feasible on local hardware.

The recreation includes a hypernetwork that generates LoRA A and B matrices from text, the L/M/S output designs, Bias-HyperInit, reconstruction from z-scored weights, and SFT through injected LoRA adapters. It targets the q_proj and v_proj modules with rank 8. Unlike the original paper, the implementation described below uses the all-MiniLM-L6-v2 sentence encoder.

The implementation follows the pipeline in Figure 1 and uses `run.py` as its command-line entry point. The original implementation was used as a reference for the method, while the project code was organized as a separate implementation.

3.2 Implementation overview

The implementation outlined in Figure 1 follows the same high-level pipeline as the paper:

Task description $\xrightarrow{f}$ text embedding
$\oplus$ layer and module embeddings (plus A/B and rank embeddings for M/S)
$\rightarrow$ residual MLP backbone $\rightarrow$ output head $\rightarrow$ LoRA A, B
$\rightarrow$ frozen language model with injected LoRA $\rightarrow$ L1 loss (reconstruction) or cross-entropy loss (SFT)

Figure 1: Overview of the T2L pipeline implemented in `t2l/model.py`.

Model and hardware The base model was Qwen2.5-1.5B-Instruct accessed through HuggingFace. Experiments ran on an Apple M5 Pro with 24 GB unified memory using the MPS backend and FP16 precision. Gradient checkpointing was disabled. The random seed was 42.

LoRA and Hypernetwork LoRA adapters targeted the query and value projection modules in every transformer layer. Adapters used rank $r = 8$, scaling parameter $\alpha = 16$, RSLoRA scaling, and dropout 0.05. The M hypernetwork variant was used. The `TaskEmbedder` uses the frozen all-MiniLM-L6-v2 sentence encoder from HuggingFace to map task descriptions to 384-dimensional embeddings. Task embeddings had dimension 64, module and layer embeddings each had dimension 32, the residual hidden dimension was 128, and the expanded MLP dimension was 512. `DynamicLoRALinear` replaces the target linear layers so the generated A and B weights remain connected to the gradient computation during SFT. This is important because storing the generated weights only in buffers would prevent gradients from flowing back through them to train the hypernetwork.

Training and evaluation pipeline (`run.py`). The command-line pipeline is divided into several stages:

1. `descriptions`: creates a set of natural-language descriptions for the tasks.
2. `oracles`: trains a separate PEFT LoRA adapter for each task (BoolQ, ArcEasy, PIQA, and OpenBookQA) with 300 optimizer steps using the learning rate 2×10^{-4}, batch size 1, no gradient accumulation, maximum sequence length 256, and at most 1,000 training examples. Collects the resulting A/B tensors.
3. `recon`: trains the hypernetwork to reproduce the oracle adapters using z-scored targets and text or one-hot conditioning. Early stopping is based on L1 loss on z-scored LoRA weights, up to 5,000 steps, learning rate 10^{-3}, batch size 2, and eight descriptions per task. The early-stopping threshold was 10^{-3}. The best observed loss was 0.0956.
4. `sft`: used 16 training slots and four held-out slots derived from eight unique Hugging Face classification datasets. Some datasets were repeated under separate task identifiers. Training used 2,500 optimizer steps, learning rate 5×10^{-5}, batch size 1, sequence length 192, 128 examples per task, and 16 descriptions per task.
5. `multitask` and `eval`: run baseline comparisons, evaluate zero-shot T2L, test different task descriptions, and visualize task-encoder activations with t-SNE.

4 Results

4.1 Task-encoder representation

Figure 2 shows some task-dependent structure. BoolQ and PIQA appear on opposite sides of the lower region, while ARC-Easy and OpenBookQA appear mainly in the upper region. The descriptions for each task are not uniformly mixed with descriptions from other tasks, suggesting that the encoder representations retain information about task identity. This is a qualitative visualization, however, and does not by itself show that the generated adapters will perform well.

4.2 Benchmark accuracy

Table 2 summarizes the benchmark results. Reconstruction is the strongest result from the recreation: its 77.8% average matches the oracle average and is 6.5 percentage points above the base model's average. On OpenBookQA, the reconstructed adapter scores 0.6 points above the oracle. This is consistent with the original paper's hypothesis that lossy reconstruction can sometimes act as a form of regularization. BoolQ is an exception: both adapted models score below the base model.

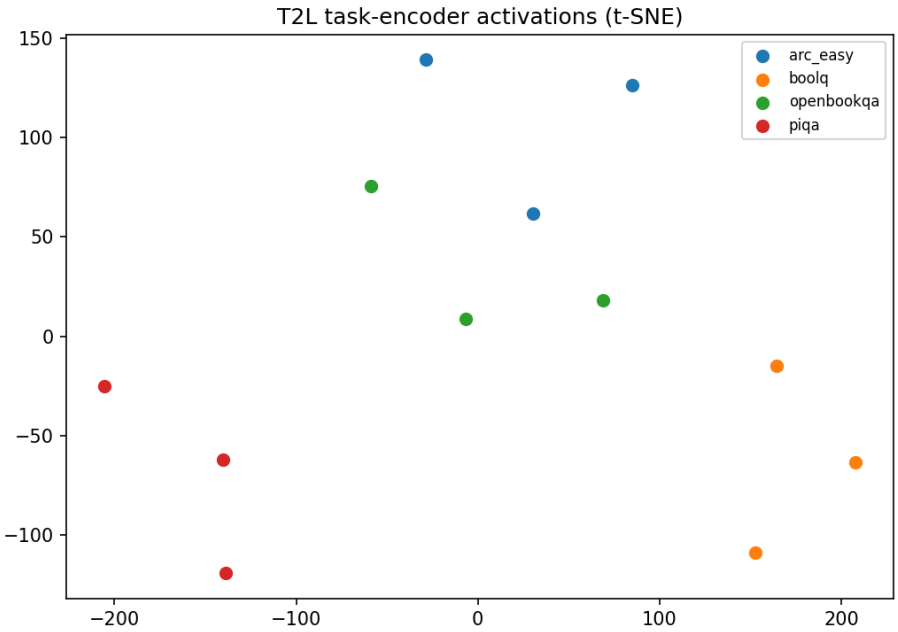


Figure 2: t-SNE projection of task-encoder activations for three descriptions of each evaluation task. The plot suggests task-dependent structure in the text representations. The axes have no intrinsic meaning

Table 2: Accuracy (%) on 500 examples per benchmark. Reconstruction closely matches the oracle results in this recreation. The initial SFT results are substantially lower and need further diagnosis.

Method	BoolQ	ARC-E	PIQA	OpenBookQA	Avg.
Base model	73.8	79.6	63.4	68.4	71.3
Oracle LoRA	69.4	90.2	74.0	77.4	77.8
Reconstructed T2L	69.0	90.2	74.0	**78.0**	**77.8**
SFT T2L (initial)	60.0	39.0	49.2	34.8	45.8

The initial SFT-trained T2L model does not show the same improvement. As shown in Table 2, its accuracy ranges from 34.8% on OpenBookQA to 49.2% on PIQA, and its four-benchmark average is 45.8%. These results do not demonstrate very successful zero-shot improvement from SFT in the current setup but scores alone do not identify the cause. Possible factors to investigate include the small training-task set, the use of synthetic default training data, the training budget, and implementation or evaluation details. However, no category scores much lower than the average which shows promise for future extensions and training for the final project.

4.3 Comparison to original paper

Table 3 places selected results beside those reported in the original paper. This is a reference point rather than a controlled comparison: the paper uses Mistral-7B-Instruct, while this recreation uses Qwen2.5-1.5B-Instruct, and the training data, compute budget, and evaluation implementation also differ. We therefore do not interpret small differences in accuracy as evidence that one method is better. We omit an overall average because the selected table includes only two benchmarks, and the broader paper reports results across a different set of tasks.

Table 3: Selected PIQA and ARC-Easy results from this project and the original paper. Original-paper reconstruction results use the medium (M) hypernetwork with task-description embeddings. Values are accuracy percentages.

Evaluation	Method	PIQA	ARC-Easy
This project			
	Base model	63.4	79.6
	Oracle LoRA	74.0	90.2
	Reconstruction T2L	74.0	90.2
	SFT-trained T2L	34.8	39.0
Original paper			
	Base model	72.8	77.8
	Oracle LoRA	69.9	89.9
	Reconstruction T2L (M)	70.4	89.9
	SFT-trained T2L (M)	81.8	89.0

4.4 Limitations of the recreation

Several differences between this implementation and the original experiments limit how directly the results can be compared:

- The base model and SFT task set are much smaller than those in the paper, which may limit the hypernetwork's ability to generalize. The final project will use a larger and more varied set of real tasks if compute permits; upgrading the base model may be impractical on local hardware.
- The task descriptions use templates and paraphrases rather than the description bank generated with GPT-4o mini in the paper. This difference may affect how much task information the descriptions convey.
- The experiments do not include full Arrow Routing or Hyperdecoder baselines; these methods are only documented in the project README.

5 Possible Extensions for the Final Project

The following extensions build on this midterm implementation and focus on task diversity, the representation of adapter weights, language-based control, model transfer, and alternative adapter designs.

5.1 Extension A: Increase task diversity under a fixed compute budget

The paper reports that adding more training tasks helps when the number of training steps also increases (Table 3 and Appendix C). We could test this by training on 8, 16, and 32 real Hugging Face classification or SNI tasks. For each task count, we would compare a fixed training-step budget with a budget that increases in proportion to the number of tasks, then plot average zero-shot benchmark performance.

Expected deliverable. Scaling curves and an analysis of whether performance is limited more by the diversity of available tasks or by insufficient training.

5.2 Extension B: Reconstruct adapters in ΔW space

The original paper notes that the individual LoRA matrices A and B can be difficult to compare directly. Different combinations of these matrices can represent similar weight updates, so comparing the matrices element by element may not always capture how similar two adapters are. The combined update, $\Delta W = BA$, may be a more useful representation of the change each adapter makes to the model.

One experiment could train the reconstruction head to predict ΔW directly, or use an SVD-based alignment of A and B, and compare the results with the current method. We could then test whether initializing SFT from the reconstructed adapters improves performance compared with training the hypernetwork from scratch.

Hypothesis. Reconstructing the actual weight update, rather than the separate factors alone, may provide a more useful starting point for transferring information from the adapter library into SFT.

5.3 Extension C: Control model behavior through task descriptions

Figure 4 in the original paper suggests that changing the wording of a task description can influence the style of the generated adapter. We could test this with controlled descriptions for the same underlying question-answering task. For example, the descriptions could request (1) a short answer, (2) a step-by-step explanation, or (3) an answer framed for an unrelated domain.

We would measure both task accuracy and whether the output follows the requested format. As an optional addition, a small local language model could rewrite descriptions before they are passed to the text encoder.

5.4 Extension D: Transfer across model sizes

An open question is whether a hypernetwork trained for one model size can generate useful adapters for a larger model in the same model family. We could train T2L using Qwen2.5-0.5B or Qwen2.5-1.5B and then evaluate whether its generated adapters transfer to Qwen2.5-3B, assuming the target layers have compatible layouts. Possible approaches include using the hypernetwork without changes or adding a small mapping layer to adjust its outputs.

Expected deliverable. A transfer matrix comparing performance across the source and target model sizes.

5.5 Recommended final-project plan

The most practical plan would be to combine Extensions A and C to test how task diversity affects performance and whether task descriptions can reliably steer model behavior. The main deliverables would be benchmark tables and t-SNE visualizations, paired with quantitative evaluation of task accuracy and output format. If time and compute permit, Extension B or D could be added. Together, these experiments would test when task descriptions help, how training-task diversity affects generalization, and whether T2L can transfer beyond its original setup.

6 Conclusion

Text-to-LoRA explores whether a model can learn to generate task-specific LoRA adapters from natural-language descriptions instead of requiring a separate adapter-training run for every task. The original paper shows that this approach can reconstruct existing adapters and, with end-to-end SFT, generate useful adapters for tasks beyond the training set.

This midterm recreation implements the main T2L components using Qwen2.5-1.5B on an Apple MacBook Pro with 24 GB of unified memory. Because the model, task set, and training budget are much smaller than those used in the paper, the goal is to study the method's behavior rather than reproduce the paper's scores exactly. The final project will investigate task scaling and description-based control, with reconstruction in ΔW space or cross-size transfer as possible extensions.

Acknowledgments and Disclosure of Funding

Course midterm project report for Deep Learning.

Method based on Charakorn et al., "Text-to-LoRA: Instant Transformer Adaption," 2025.

References

M. Beck, et al. A Survey of Meta-Reinforcement Learning. *arXiv:2301.08028*, 2023. Bias-HyperInit as used by Charakorn et al. Charakorn et al. [2025].

R. Charakorn, E. Cetin, Y. Tang, and R. T. Lange. Text-to-LoRA: Instant Transformer Adaption. *arXiv:2506.06105*, 2025. `https://arxiv.org/abs/2506.06105`.

D. Ha, A. Dai, and Q. V. Le. HyperNetworks. *ICLR*, 2017. `https://arxiv.org/abs/1609.09106`.

E. J. Hu, et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.

Y. Wang, et al. Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks. *EMNLP*, 2022.